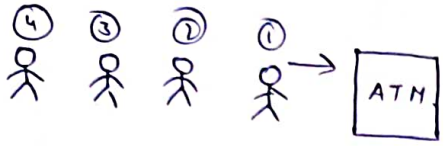
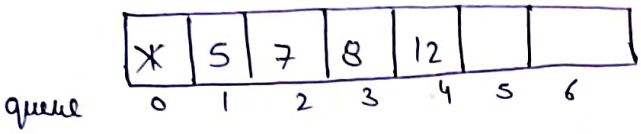
Lecture (60)

Queue

→ Type of data structure

→ FIFO concept (First in first out)





queue.push(1)

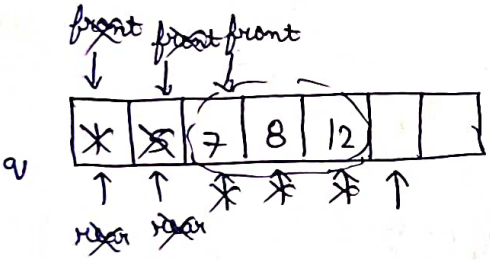
_____ (5)

_____ (7)

_____ (8)

_____ (12)

queue.pop() → FIFO ↷ removes 1



1, 5, 7, 8, 12

push → rear

pop → front

For creation ⇒ queue<int> q;

push ⇒ q.push(17);

pop ⇒ q.pop();

size ⇒ q.size();

is empty ⇒ q.empty(); — T/F

Sabse aage element kon sa ⇒ q.front(); last element ⇒ q.back();

Qu- Implement a queue with array.

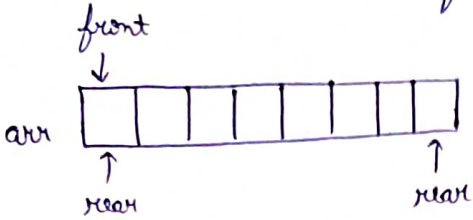
class Queue

↳ Data members

↳ arr

↳ size

↳ front / rear



if (rear == n) → full

push

↳ if (full) → Queue is full

↳ else → arr [rear] = element;

rear ++;

empty

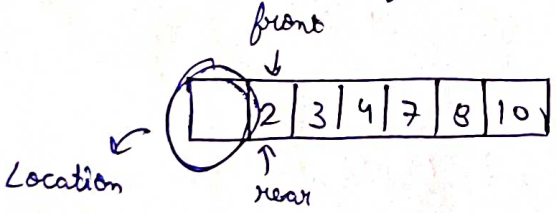
↳ front == rear

pop

↳ if (empty) → Queue is empty

↳ else → arr [front] = -1;

front++;




```
if (front == rear) {
    front = 0;
    rear = 0;
}
```

isEmpty

↳ if (front == rear) → empty

front()

↳ if (empty) → return -1

↳ else → return arr [front];

```
Class Queue {
    int * arr;
    int qfront;
    int rear;
    int size;
public:
    Queue (int size) {
        this -> size = size;
        arr = new int [size];
        qfront = 0;
        rear = 0;
    }
    void enqueue (int data) {
        if (rear == size) {
            cout << "Queue is full" << endl;
        }
        else {
            arr [rear] = data;
            rear++;
        }
    }
    int dequeue () {
        if (qfront == rear) {
            return -1;
        }
        else {
            int ans = arr [qfront];
            arr [qfront] = -1;
```

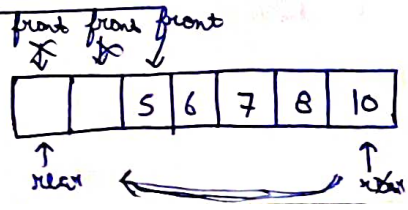
```
        qfront++;
        if (qfront == rear) {
            qfront = 0;
            rear = 0;
        }
        return ans;
    }
}

int front() {
    if (qfront == rear) {
        return -1;
    }
    else {
        return arr[qfront];
    }
}

bool isEmpty() {
    if (qfront == rear) {
        return true;
    }
    else {
        return false;
    }
}
};
```

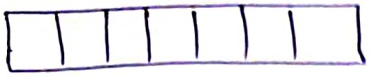
T.C = O(1)

Circular Queue



Qu- Implement a Circular Queue

Push



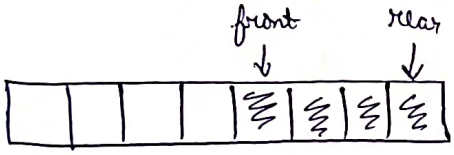
front = -1;
rear = -1;

① if (full) → "Queue is full";

full
↳ (front == 0 && rear == size-1)
↳ (rear == (front-1) % (size-1)
if (T)

② if (front == -1) // (first element)
↳ front = rear = 0;
arr [queue] = data;

③

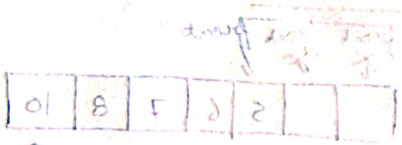


if (rear == size-1 && front != 0)
↳ rear = 0
arr [rear] = data;
else
↳ rear++;
arr[rear] = data;

Pop

① if (empty) → "Queue is empty"
↳ front == -1;

② // single element in queue
if (front == rear)
↳ front = rear = -1;



③ if (front == size-1)

↳ front=0;

④ Normal case

↳ front++;

```
class CircularQueue {
    int * arr;
    int front;
    int rear;
    int size;
    public:
    CircularQueue (int n) {
        size = n;
        arr = new int [size];
        front = rear = -1;
    }
    bool enqueue (int value) {
        if ((front==0 && rear== size-1) || (rear == (front-1) %(size-1))){
            return false;
        }
        else if (front ==-1) {
            front = rear =0;
        }
        else if (rear == size-1 && front != 0) {
            rear = 0;
        }
        else {
            rear++;
        }
```

```
        arr[rear] = value;
        return true;
    }
    int dequeue() {
        if (front == -1) {
            return -1;
        }
        int ans = arr[front];
        arr[front] = -1;
        if (front == rear) {
            front = rear = -1;
        }
        else if (front == size - 1) {
            front = 0;      // To maintain cyclic nature
        }
        else {
            front++;
        }
        return ans;
    }
};
```

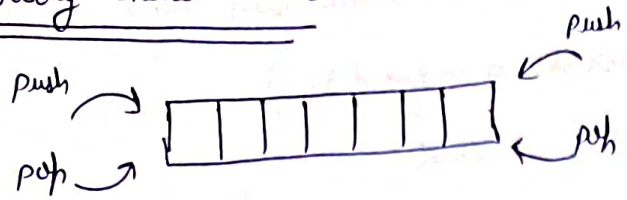
Input restricted Queue → o/p → both side possible

- push_back
- pop_front
- pop_back

Output restricted Queue → i/p → both side possible

- push_front
- push_back
- pop_front

Doubly ended Queue



can be pushed & popped from both sides

Qu- Implement a doubly ended queue (Deque)

```
front = -1;
rear = -1;
```

push front

↳ if (full) → Queue is full

↳ // first element insert

if (front == -1)

↳ front = rear = 0

↳ // Cyclic nature maintain

if (front == 0)

↳ front = n-1

↳ //normal flow

↳ front --

pop front

↳ same logic

push rear

↳ same as Circular Queue

pop front ↳ same as Circular Queue

pop back

↳ if (empty) → Queue is empty

↳ // Single element

↳ front = rear = -1

↳ // Cyclic nature maintain

if (rear == 0) ↳ rear = n-1

↳ normal flow

↳ rear --

```
class Deque {
    int *arr;
    int front;
    int rear;
    int size;

public:
    Deque(int n) {
        size = n;
        arr = new int[n];
        front = -1;
        rear = -1;
    }

    bool pushFront(int x) {
        if ((front == 0 && rear == size-1) || (rear == (front-1) %
                                                (size-1))) {
            return false;
        }
        else if (front == -1) {
            front = rear = 0;
        }
        else if (front == 0 && rear != size-1) {
            front = size-1;
        }
        else {
            front--;
        }
        arr[front] = x;
        return true;
    }
```

```
bool pushRear (int x) {
    if ((front == 0 && rear == size-1) || (rear == (front-1) % (size-1))) {
        return false;
    }
    else if (front == -1) {
        front = rear = 0;
    }
    else if (rear == size-1 && front != 0) {
        rear = 0;
    }
    else {
        rear++;
    }
    arr [rear] = x;
    return true;
}
int popFront() {
    if (front == -1) {
        return -1;
    }
    int ans = arr [front];
    arr [front] = -1;
    if (front == rear) {
        front = rear = -1;
    }
    else if (front == size-1) {
        front = 0;
    }
```

```
        else {
            front++;
        }
        return ans;
    }

    int popRear() {
        if (front == -1) {
            return -1;
        }
        int ans = arr[rear];
        arr[rear] = -1;
        if (front == rear) {
            front = rear = -1;
        }
        else if (rear == 0) {
            rear = size - 1;
        }
        else {
            rear--;
        }
        return ans;
    }

    int getFront() {
        if (isEmpty()) {
            return -1;
        }
```

```
        return arr[front];
    }

    int getRear() {
        if (isEmpty()) {
            return -1;
        }
        return arr[rear];
    }

    bool isEmpty() {
        if (front == -1) {
            return true;
        }
        else {
            return false;
        }
    }

    bool isFull() {
        if ((front == 0 && rear == size-1) || (rear == (front-1)%(size-1)))
            return true;
        }
        else {
            return false;
        }
    }
};
```

Lecture (61)

Qu- Queue Reversal

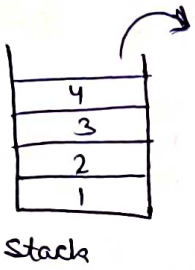
i/p ⇒ 4 3 1 10 2 6

o/p ⇒ 6 2 10 1 3 4

Approach (1)

↳ By using stack

queue(1, 2, 3, 4)



queue

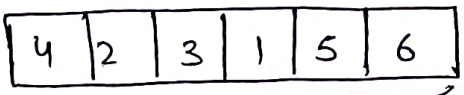
T.C. = O(n)

S.C. = O(n)

Approach (2)

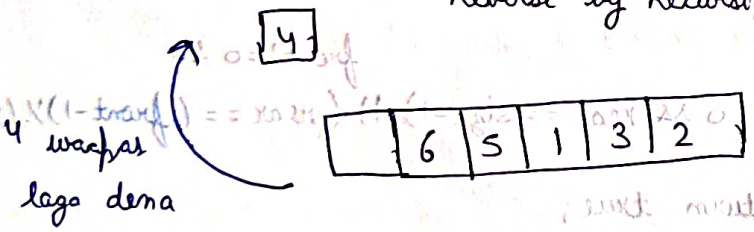
↳ By Recursion

i/p ⇒ q



Reverse by Recursion

T.C. = O(n)



Code (1)

```
queue<int> rev(queue<int> q) {
    stack<int> s;
    while(!q.empty()) {
        int element = q.front();
        q.pop();
        s.push(element);
    }
```

```
while (!s.empty()) {
        int element = s.top();
        s.pop();
        q.push(element);
    }
    return q;
}
```

Qu- First negative integer in every window of size K.

i/p => A[] = {-8, 2, 3, -6, 10}
K = 2

o/p => -8, 0, -6, -6

My sol^n

TLE

```
queue<int> q;
int i=0;
int m = K;
while (i<K && i<=n-m) {
    for(int j=i; j<K; j++) {
        if (arr[j] <0) {
            q.push(arr[j]);
            break;
        }
        else if (j==K-1 && arr[j] >= 0) {
            q.push(0);
            break;
        }
    }
    K++;
    i++;
}
```

Approach

deque < int > dq ;

① first K element (first window)

answer ⇒

② Agli window me ek naya no. add ho raha hai aur ek remove ho raha hai


```
Vector < long long > printFirstNegativeInteger ( long long int A[], long long
                                                 int N, long long int K) {

    deque < long long int > dq ;
    Vector < long long > ans;
    // Process first window of K size
    for (int i=0; i<K; i++) {

        if (A[i] < 0) {

            dq.push_back(i);

        }

    }

    // Store answer of first K sized window
    if ( dq.size() > 0) {

        ans.push_back (A[dq.front()]);

    else {

        ans.push_back (0);

    }

    // Process for remaining windows
```

```
for (int i=K; i<N; i++) {

    //removal
    if (!dq.empty() && i-dq.front() >= K) {
        dq.pop_front();
    }

    // addition
    if (A[i] < 0) {
        dq.push_back(i);
    }

    // ans store
    if (dq.size() > 0) {
        ans.push_back(A[dq.front()]);
    }
    else {
        ans.push_back(0);
    }
}
return ans;
}
```

H.W.

Optimal approach of this question

Qu- Reverse first K elements of queue

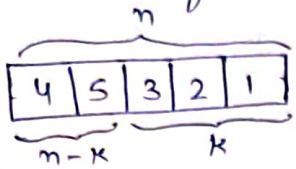
i/p ⇒ {1, 2, 3, 4, 5} (1, 2, 3: reverse) K=3

o/p ⇒ {3, 2, 1, 4, 5}

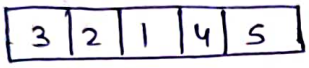
Algo:-

① Fetch first K element from queue & put into stack

② Fetch element from stack & put into queue.



③ Fetch first (n-k) elements from queue & push_back.




```
queue<int> modifyQueue(queue<int> q, int k) {
    Stack<int> s;
    for(int i=0; i<k; i++) {
        int val = q.front();
        q.pop();
        s.push(val);
    }
    while(!s.empty()) {
        int val = s.top();
        s.pop();
        q.push(val);
    }
    int t = q.size() - k;
    while(t--) {
        q.push(q.front());
        q.pop();
    }
    return q;
}
```

S.C. = O(K)

T.C. = O(n)

Qu- First non-repeating character in a stream, return # if there is no such character

i/p => a a b c

o/p => a # b b

i/p => z z

o/p => z #

Approach

① For storing count of each char => map<char, int> count;

② a a b c

(i) count[a]++;

(ii) q.push(a);

q.front() → Repeating → pop();

→ Non-repeating → count/ans store

→ empty → return #

```
string FirstNotRepeating(string A) {
    unordered_map<char, int> count;
    queue<int> q;
    string ans = "";
    for(int i=0; i<A.length(); i++) {
        char ch = A[i];
        //increase count
        count[ch]++;
        // queue me push karo
        q.push(ch);
        while(!q.empty()) {
            if(count[q.front] > 1) {
                //repeating char
                q.pop();
```

```
        else {
            //non repeating char
            ans.push_back(q.front());
            break;
        }
    }
    if (q.empty()) {
        ans.push_back('#');
    }
}
return ans;
}
```

Qu- Suppose there is a circle. There are n petrol pumps on that circle. You will be given two sets of data:

(i) the amount of petrol that every petrol pump has

(ii) distance from that petrol pump to the next petrol pump.

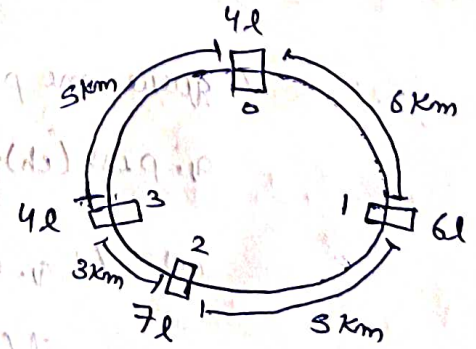
Find a starting point where a truck can start to get through the complete circle without exhausting it's petrol in between.

Assume for 1 litre petrol, the truck can go 1 unit of distance.

i/p ⇒ n = 4

Petrol = 4 6 7 4

Distance = 6 5 3 5

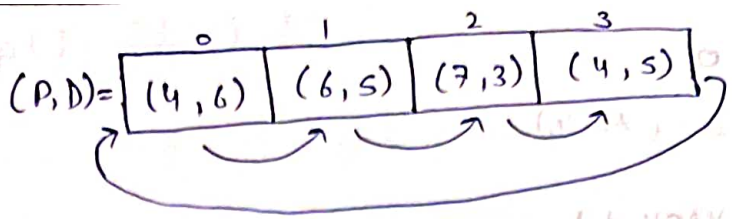


o/p ⇒ 1

(P, D) =

(4, 6)	(6, 5)	(7, 3)	(4, 5)
0	1	2	3

Approach



balance ⇒ P - D

if start = 0

balance = P - D

= 4 - 6 = -2 (Not possible)

if start = 1

balance = 6 - 5 = 1 >= 0

1 + 7 = 8 l

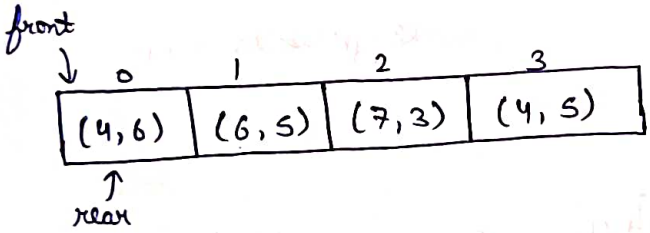
balance = 8 - 3 = 5 >= 0

5 + 4 = 9 l

balance = 9 - 5 = 4 >= 0

4 + 4 = 8 l

balance = 8 - 6 = 2 >= 0 (Possible)



Algo:

if (travel possible) i.e P - D >= 0

↳ rear ++ ;

if (front == rear)

↳ circle complete

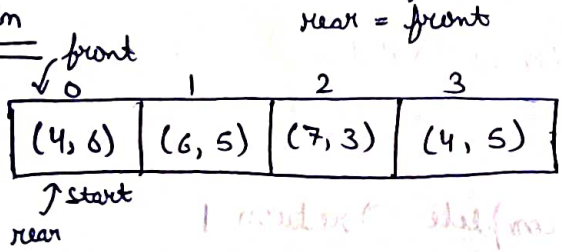
But (if travel not possible)

↳ front = rear + 1
start = front
rear = front

(Not front ++ X)

↳ Kyoki wo contribution ke saath nhi pahuch paa rahe To without contribution to nhi hi pahuchega

Dry Run

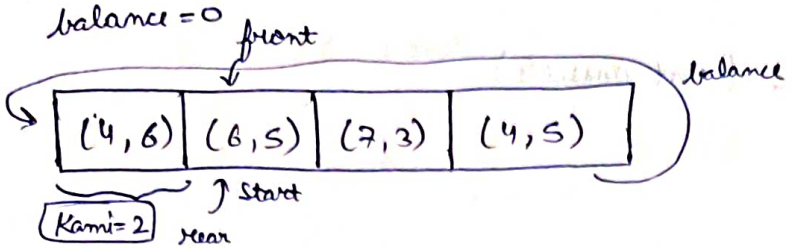


① $P - D >= 0$

$4 - 6 = -2$ (False)

front = rear + 1

balance = 0



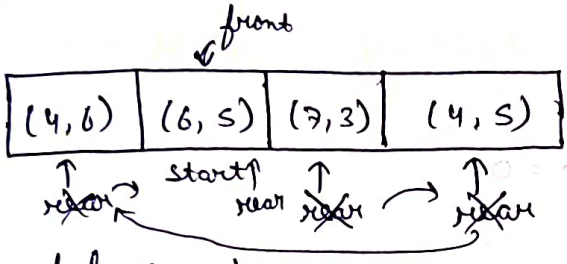
(balance + kami >= 0)

↳ and possible

② $P - D >= 0$

$6 - 5 = 1 >= 0$ (True)

rear ++;



balance = 1

$P = 1 + 7 = 8$

$P - D >= 0$

$8 - 3 = 5 >= 0$ (True)

rear ++;

balance = 5

$P = 5 + 4 = 9$

$P - D >= 0$

$9 - 5 = 4 >= 0$ (True)

rear ++;

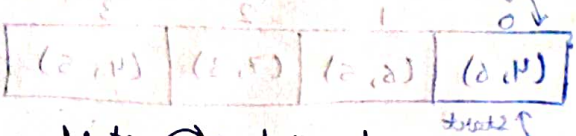
balance = 4

$P \leq 4 + 4 = 8$

$P - D >= 0$

$8 - 6 = 2 >= 0$ (True)

rear ++;



Now, front == rear ↝ Cycle complete ↝ return 1

```
int tour(petrolPump p[], int n) {
    int deficit = 0;
    int balance = 0;
    int start = 0;
    for (int i=0; i<n; i++) {
        balance += p[i].petrol - p[i].distance;
        if (balance < 0) {
            deficit += balance;
            start = i+1;
            balance = 0;
        }
    }
    if ((deficit + balance) >= 0) {
        return start;
    }
    else {
        return -1;
    }
}
```

H.W.

Qu- Interleave the first half of the queue with second half

i/p ⇒ {11, 12, 13, 14 | 15, 16, 17, 18}

o/p ⇒ {11, 15, 12, 16, 13, 17, 14, 18}

Approach ①

① Fetch first half element from i/p queue & push into a new queue

q = | 15 | 16 | 17 | 18 |

new q = | 11 | 12 | 13 | 14 |

② while (! new q . empty()) {

```
int val = newq.front();
newq.pop();
q.push(val);
val = q.front();
q.pop();
q.push(val);
```

}

T.C. = O(n)

S.C. = O(n)

H.O.

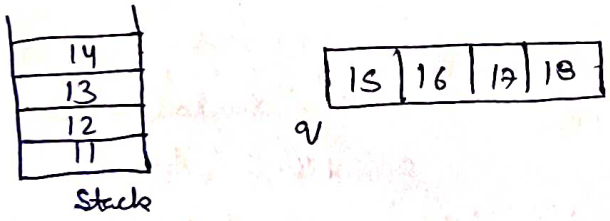
① Implement Stack using Queue

② Implement Queue using Stack

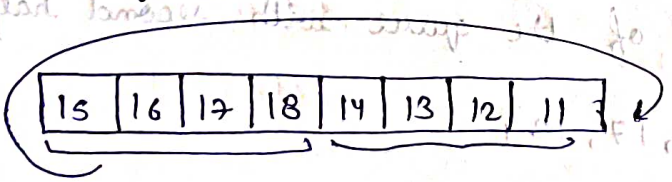
Approach ②

{11, 12, 13, 14, 15, 16, 17, 18}

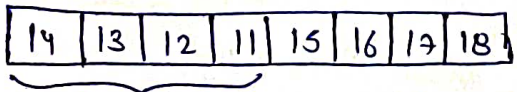
① First half of q → stack



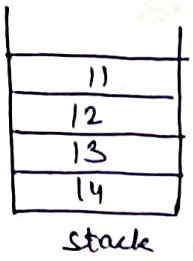
② stack → queue



③ First half of q pop & push at back of q



④ First half of q → Stack



q {15, 16, 17, 18}

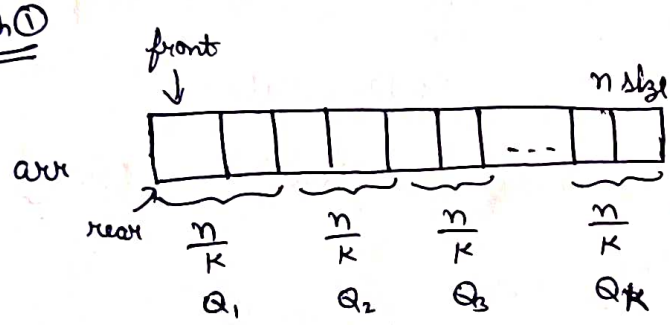
⑤ while (!s.empty()) {

```
int val = s.top();
s.pop();
q.push(val);
val = q.front();
q.pop();
q.push(val);
```

}

Qu- Implement 'K' queues in an array

Approach①

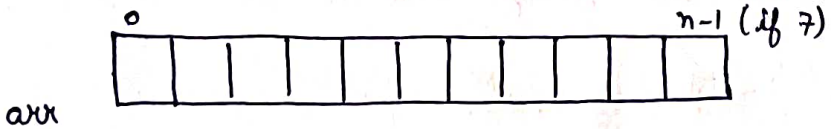


Divide in K parts

1 part → n/K

↳ not space optimized

Approach②



front[K] ⇒ | -1 | -1 | -1 |

rear[K] ⇒ | -1 | -1 | -1 |

} if (K=3) for example

next[n] ⇒

0	1	2	3	4	5	6	7
1	2	3	4	5	6	7	-1

free slot ⇒ 0

push

↳ Overflow check (when freeslot = -1)

↳ // find index where we want to insert

```
int index = freeSpot;
```

↳ // update freeSpot

```
freeSpot = next[index];
```

↳ // if first element

```
if (front[qs] == -1) {
    front[qs] = index;
}
else {
    next[rear[qs]] = index;
}
```

```
class kQueue {

public:
    int n;
    int k;
    int *front;
    int *rear;
    int *arr;
    int freeSpot;
    int *next;

    kQueue(int n, int k) {
        this -> n = n;
        this -> k = k;
        front = new int[k];
        rear = new int[k];
```

```
for (int i=0; i<K; i++) {
        front[i] = -1;
        rear[i] = -1;
    }
    next = new int[n];
    for (int i=0; i<n; i++) {
        next[i] = i+1;
    }
    next[n-1] = -1;
    arr = new int[n];
    freeSpot = 0;
}
void enqueue (int data, int qn) {
    // overflow
    if (freeSpot == -1) {
        cout << "no empty space is present" << endl;
        return;
    }
    // find first free index
    int index = freeSpot;
    // update freeSpot
    freeSpot = next[index];
    // check whether first element
    if (front[qn-1] == -1) {
        front[qn-1] = index;
    }
```

```
    else {
        // link new element to the prev element
        next[rear[qn-1]] = index;
    }

    // update next
    next[index] = -1;

    // update rear
    rear[qn-1] = index;

    // push element
    arr[index] = data;
}

void dequeue(int qn) {
    // underflow
    if (front[qn-1] == -1) {
        cout << "Queue Underflow" << endl;
        return;
    }

    // find index to pop
    int index = front[qn-1];

    // front ko age badhao
    front[qn-1] = next[index];

    // free slots ko manage karo
    next[index] = freeSpot;
    freeSpot = index;
    return arr[index];
}
};
```

Qu- Sum of minimum and maximum elements of all subarrays of size k.

i/p ⇒ {2, 5, -1, 7, -3, -1, -2} k = 4

o/p ⇒ 18

Explanation :-

{2, 5, -1, 7} ⇒ -1 + 7 = 6
{5, -1, 7, -3} ⇒ -3 + 7 = 4
{-1, 7, -3, -1} ⇒ -3 + 7 = 4
{7, -3, -1, -2} ⇒ -3 + 7 = 4

6 + 4 + 4 + 4 = 18

Approach

{2, 5, -1, 7, -3, -1, -2}

K-size

① deque (mini) → for tracking min. element (decreasing order me element honge)

② deque (maxi) → for tracking max. element (decreasing order me element honge)

```
int solve ( int * arr, int K, int n) {

    deque <int> maxi(K);
    deque <int> mini(K);

    // Addition of first K size window
    for (int i=0; i<K; i++) {

        while ( arr[maxi.back()] <= arr[i] && !maxi.empty() ) {
            maxi.pop_back();
        }

        while ( arr[mini.back()] >= arr[i] && !mini.empty() ) {
            mini.pop_back();
        }

        maxi.push_back(i);
        mini.push_back(i);
    }
```

T.C. = O(n)

```
int ans = 0;
for (int i = k; i < n; i++) {
    ans += arr[maxi.front()] + arr[mini.front()];

    while (!maxi.empty() && i - maxi.front() >= k) {
        maxi.pop_front();
    }

    while (!mini.empty() && i - mini.front() >= k) {
        mini.pop_front();
    }

    while (!maxi.empty() && arr[maxi.back()] <= arr[i]) {
        maxi.pop_back();
    }

    while (!mini.empty() && arr[mini.back()] >= arr[i]) {
        mini.pop_back();
    }

    maxi.push_back(i);
    mini.push_back(i);
}
ans += arr[maxi.front()] + arr[mini.front()];
return ans;
}
```